CS 4510 Automata and Complexity 4/13/2025

Homework 10G: Savitch's and Non-deterministic Hierarchy Theorem

Devin Fromond *Due:4/23/2025*

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments. Four bonus points if you use LaTeX, and our template. You may collaborate with other students, but any written work should be your own. **Many of these problems have two-sentence answers if you can apply the theorems proved in class**.

1. (10 points) **Savitch's Theorem.**
   Savitch's Theorem states that for any space-constructable function $f(n) \geq \log n$,

$$\text{NSPACE}\big(f(n)\big) \subseteq \text{DSPACE}\big(f(n)^2\big).$$

   (a) (6 points) Prove that any language L in NSPACE(n) also lies in DSPACE(n^2). Show the recursive "configuration reachability" approach that achieves this.
   Answer: We wish to define an M such that it is an $O(n)-space$ non-deterministic Turing Machine. We find each configuration is specified by its state, tape contents on $\leq cn$ cells, and head position. Relatively simply, we find the total possible configurations is $N = 2^{O(n)}$.
   To deterministically accept in $O(n^2)$ space, we wish to define a *recursive configuration reachability routine* defined as $Reach(c_1, c_2, t)$ that will check if the configuration c_2 is reachable from c_1 in $\leq t$ steps. As a direct application of Savitch's Theorem, we find if $t = 1$, then we simulate one more move. Otherwise, we split t into half, enumerate all N mid-configurations m, and accept if for some m, both $Reach(c_1, m, \frac{t}{2})$ and $Reach(m, c_2, \frac{t}{2})$ accepts. We observe each call stores $O(n)$ bits and the depth of the recursion call is $O(log(N)) = O(n)$, so the total space is $O(n^2)$.

   (b) (4 points) Let M be a nondeterministic Turing machine that uses $O(n)$ space on inputs of length n. Estimate the total number of configurations of M and then deduce an upper bound (in terms of n) on the worst-case time complexity of the deterministic simulation.
   Answer: We observe M uses at most $O(n)$ tape cells. Similarly, we find each configuration is determined by its current state (which is one of $|Q|$ choices), the contents of the $O(n)$ cells, and the head position (one of the $O(n)$ spots). Therefore, we find the following:

$$N \leq |Q| \cdot |\Gamma|^{O(n)} \cdot O(n) = 2^{O(n)}$$

   We observe the deterministic reachability algorithm as previously described makes 2 recursive calls for each of N midpoints for each level with a max depth of

$O(log(N)) = O(n)$. Since the machine is fixed, we recognize $|Q|$ and $|\Gamma|$ are constants. Therefore, with a resemblance to the Master's T^{hm} (shoutout CS 3510), we find the following:

$$T(N) = N[2T(N/2) + O(1)] = N^{O(log(N)} = 2^{O(n^2)}$$

2. (30 points) **Non-deterministic Time Hierarchy Theorem.**

The NTIME Hierarchy Theorem says that for every proper complexity function $T(n) \geq n$ and any $T'(n)$ satisfying $T(n+1) = o\big(T'(n)\big)$,

$$\text{NTIME}\big(T(n)\big) \subsetneq \text{NTIME}\big(T'(n)\big).$$

(a) (10 points) Suppose

$$T_1(n) = 2^{n^2} \quad \text{and} \quad T_2(n) = n^4 \cdot 2^n.$$

Which of the following is correct?

(a) $\text{NTIME}\big(T_1(n)\big) = \text{NTIME}\big(T_2(n)\big)$
(b) $\text{NTIME}\big(T_1(n)\big) \subsetneq \text{NTIME}\big(T_2(n)\big)$
(c) $\text{NTIME}\big(T_2(n)\big) \subsetneq \text{NTIME}\big(T_1(n)\big)$

Give a justification of your answer based on comparing 2^{n^2} and $n^4 \cdot 2^n$ and use NTIME Hierarchy Theorem to support your conclusion.

Answer: We observe $\frac{T_2(n)}{T_1(n)} = \frac{n^4 2^n}{2^{n^2}}$ approaches zero as n approaches infinity. Therefore, by definition, we find $T_2(n) = o(T_1(n))$. Similarly, we observe $T_2(n+1) = (n+1)^4 2^{n+1} = 2(n+1)^4 2^n = o(2^{n^2}) = o(T_1(n))$. Therefore, by utilizing the NTIME Hierarchy Theorem, we find the following truths:

$$T(n) = T_2(n)$$

$$T'(n) = T_1(n)$$

$$T(n+1) = o(T'(n))$$

Therefore, directly from the Theorem, we find $NTIME(T_2(n)) \subset NTIME(T_1(n))$. Therefore, we find the correct choice is (c).

(b) (5 points) In one or two sentences, explain how the condition $T(n+1) = o\big(T'(n)\big)$ ensures a strict separation (i.e., $\subsetneq$) between $\text{NTIME}\big(T(n)\big)$ and $\text{NTIME}\big(T'(n)\big)$.

Answer: The condition ensures that after shifting the input-length index by one, T grows strictly slower than T'. The gap between the two growths allows the diagonalization argument to construct a language that cannot be decided within T but is decidable within T', resulting in a strict inclusion.

(c) (5 points) Prove that any proper complexity function $T(n)$ is non-decreasing. Briefly explain why this property is required in the context of the NTIME Hierarchy Theorem.

Answer: We wish to define a non-decreasing version of T such that $T^*(n) = \max_{1 \le i \le n} T(i)$. Since T is time-constructible, we observe we may compute all $T(1), \ldots, T(n)$ in $O(T(n))$ time and therefore compute $T^*(n)$ within $O(T(n))$ steps. Relatively simply, we observe $T^*(n) \ge T(n) \ge n$, $T^*(n) = \Theta(T(n))$, and that T^* is non-decreasing. Therefore, we observe we may replace T by T^* without changing its asymptotic class or constructibility. Monotonicity is necessary for diagionalization so that the intervals are disjoint and nested, ensuring each M_i has a block of input that does not overlap with other blocks.

(d) (10 points) Refer to the diagonalization strategy discussed in lecture: intervals $[1 \ldots T(1)]$ "kill" M_1, intervals $[T(1) \ldots T(T(1))]$ kill M_2, etc. Describe how this strategy constructs a nondeterministic Turing machine D that differs from every M_i in $\text{NTIME}\big(T(n)\big)$. Also, explain why D still runs within $\text{NTIME}\big(T'(n)\big)$.

Answer: On the unary input 1^n, D first finds the smallest i such that $T^{i-1}(1) < n \le T^i(1)$ and sets $k = T^{i-1}(1) + 1$. D then non-deterministically guesses one accepting/rejecting branch of M_i on input 1^k, simulates that branch for at most $T(k) \le T(n)$ steps, and flips the answer (similarly to the deterministic version). Therefore, we find D disagrees with M_i on at least one string of length k. We observe computing i and k takes $O(n) \subseteq O(T(n))$, so the total non-deterministic time of D is $O(T(n))$. Since $T(n+1) = o\big(T'(n)\big)$, for sufficiently large n we have $T(n) + n \le T(n+1) = o\big(T'(n)\big) \le T'(n)$, so $D \in \text{NTIME}\big(T'(n)\big)$.